

Programación Estructurada en Python

De la Sintaxis Fundamental al Cálculo Matricial con NumPy

Manual exhaustivo con explicaciones detalladas, derivaciones analíticas, código fuente funcional y salidas reales por terminal.

Módulos Incluidos en este Documento

- 1. F-Strings: Interpolación de cadenas y formato numérico avanzado.
- 2. Definición de Funciones: Parámetros (*args, **kwargs), docstrings y tipado.
- 3. Cálculos Geométricos: Áreas de rectángulo y triángulo (estándar y Herón).
- 4. Volúmenes de Sólidos: Cilindro, esfera, prisma rectangular y cono.
- 5. Creación y Manipulación de Listas: Métodos nativos y comprensiones de listas.
- 6. Manipulación de Strings: Saneamiento, segmentación, unión y búsqueda.
- 7. Control de Flujo con if: Operadores lógicos, relacionales y ternarios.
- 8. Bucles y Patrones con for i: Rangos, anidación y figuras geométricas.
- 9. Paquete NumPy: Arrays n-dimensionales, álgebra lineal y vectorización.

Documento Técnico Compilado en L^AT_EX

Guía de referencia formativa y profesional para Python 3.x

Índice

1. Cadenas F-Strings (Formato Literal de Cadenas)	1
1.1. Sintaxis e Interpolación Básica	1
1.2. Especificadores de Formato Numérico y Alineación	1
2. Definición y Modularización de Funciones (def)	3
2.1. Parámetros por Defecto y Retorno de Múltiples Valores	3
2.2. Parámetros Arbitrarios: *args y **kwargs	3
3. Cálculos Geométricos Básicos: Área de Rectángulo y Triángulo	5
3.1. Área de un Rectángulo	5
3.2. Área de un Triángulo	5
4. Cálculo del Volumen de Sólidos Geométricos	7
5. Creación, Métodos y Manipulación de Listas	9
5.1. Operaciones Fundamentales y Métodos Nativos	9
5.2. Comprensión de Listas (<i>List Comprehensions</i>)	9
6. Manipulación y Procesamiento de Cadenas de Texto (<i>Strings</i>)	11
6.1. Métodos Principales de Procesamiento	11
7. Estructuras de Control Condicional (if, elif, else)	13
7.1. Operadores Relacionales y Lógicos	13
8. Bucles Iterativos y Generación de Patrones con for i	14
8.1. Generación de Patrones con Bucles Anidados	14
9. Computación Científica y Matricial con NumPy	16
9.1. Creación y Estructura de Arreglos (ndarray)	16
9.2. Operaciones Vectorizadas y Álgebra Lineal	16
9.3. Indexación Avanzada, Slicing y Máscaras Booleanas	18
9.4. Cálculos Estadísticos y de Reducción	18
10. Resumen y Tabla de Referencia Rápida	19

1. Cadenas F-Strings (Formato Literal de Cadenas)

Introducidas en Python 3.6 a través del PEP 498, las **f-strings** (*formatted string literals*) constituyen el método estándar, más eficiente y legible para realizar interpolación de variables y evaluación de expresiones dinámicas.

1.1. Sintaxis e Interpolación Básica

Se define una f-string anteponiendo el carácter **f** o **F** antes de las comillas que abren la cadena. Cualquier variable o expresión encerrada entre llaves **{ }** se evalúa directamente:

Interpolación y evaluación con f-strings

```
1 nombre = "Carlos"
2 edad = 23
3 altura = 1.7854
4
5 # Interpolación de variables
6 mensaje = f"Hola, mi nombre es {nombre} y tengo {edad} años."
7 print(mensaje)
8
9 # Evaluación directa de expresiones matemáticas
10 print(f"El doble de mi edad dentro de 5 años será: {(edad + 5) * 2}")
```

Salida en Consola

```
Hola, mi nombre es Carlos y tengo 23 años.
El doble de mi edad dentro de 5 años será: 56
```

1.2. Especificadores de Formato Numérico y Alineación

Las f-strings soportan el mini-lenguaje de especificación de formato de Python tras dos puntos **::**:

- **{valor:.2f}**: Redondea un número flotante a 2 cifras decimales.
- **{valor:,.2f}**: Aplica comas como separador de miles y 2 decimales.
- **{valor:.1%}**: Multiplica por 100 y formatea como porcentaje.
- **{valor:>15}** / **{valor:<15}** / **{valor:^15}**: Alineación a la derecha, izquierda o centro con ancho fijo.
- **{variable=}**: Imprime la expresión y su valor para depuración rápida (Python 3.8+).

Formateo numérico y alineación tabular

```
1 precio = 1254300.6789
2 tasa = 0.0525
3 producto = "Laptop Gamer"
4
5 print(f"Producto: {producto:<20} | Precio: ${precio:,.2f}")
6 print(f"Tasa de interés anual: {tasa:.1%}")
7
8 # Depuración directa (debug mode)
```

```
9 x = 42
10 y = 17
11 print(f"{x + y = }")
```

Salida en Consola

```
Producto: Laptop Gamer      | Precio: $1,254,300.68
Tasa de interés anual: 5.2%
x + y = 59
```

2. Definición y Modularización de Funciones (def)

Las funciones son bloques de código encapsulado y reutilizable diseñados para llevar a cabo una tarea concreta. Se declaran mediante la palabra clave **def**, seguida del identificador, los argumentos entre paréntesis y dos puntos **:**.

Buenas Prácticas en el Diseño de Funciones

1. **Modularidad:** Dividir programas extensos en submódulos comprensibles y testeables.
2. **Documentación (Docstrings):** Utilizar cadenas de triple comilla `"""` para documentar parámetros, precondiciones y valores de retorno.
3. **Anotaciones de Tipo (Type Hints):** Especificar tipos de entrada y salida para mejorar la legibilidad y detección temprana de errores.

2.1. Parámetros por Defecto y Retorno de Múltiples Valores

Python permite asignar valores predeterminados y devolver tuplas que pueden desempaquetarse directamente:

Definición con docstrings y retornos múltiples

```
1 def calcular_estadisticas(valores: list[float], redondear: bool = True
2     ) -> tuple[float, float, float]:
3     """
4     Calcula el mínimo, el máximo y la media aritmética de una lista
5     numérica.
6     """
7     if not valores:
8         raise ValueError("La lista de valores no puede estar vacía.")
9
10    val_min = min(valores)
11    val_max = max(valores)
12    promedio = sum(valores) / len(valores)
13
14    if redondear:
15        return round(val_min, 2), round(val_max, 2), round(promedio,
16            2)
17    return val_min, val_max, promedio
18
19 # Invocación y desempaqueado de tuplas
20 datos = [14.5, 22.1, 8.9, 31.0, 17.4]
21 minimo, maximo, prom = calcular_estadisticas(datos)
22 print(f"Mínimo: {minimo} | Máximo: {maximo} | Promedio: {prom}")
```

Salida en Consola

```
Mínimo: 8.9 | Máximo: 31.0 | Promedio: 18.78
```

2.2. Parámetros Arbitrarios: *args y **kwargs

- ***args:** Agrupa un número indeterminado de argumentos posicionales en una tupla.

- ****kwargs**: Agrupa argumentos nombrados (*keyword arguments*) en un diccionario.

Uso de *args y **kwargs

```
1 def procesar_orden(cliente: str, *articulos: str, **detalles) -> None:
2     print(f"--- Orden de Compra: {cliente} ---")
3     print(f"Total de artículos: {len(articulos)}")
4     for i, art in enumerate(articulos, start=1):
5         print(f"  {i}. {art}")
6     print("Detalles del envío:")
7     for clave, valor in detalles.items():
8         print(f"  - {clave.capitalize()}: {valor}")
9
10 procesar_orden("Lucía Gómez", "Monitor 4K", "Teclado", "Mouse",
11                descuento="10%", entrega="Exprés", ciudad="Bogotá")
```

3. Cálculos Geométricos Básicos: Área de Rectángulo y Triángulo

En esta sección se abordan las fórmulas geométricas planas y su correspondiente implementación en Python con validación de entradas.

3.1. Área de un Rectángulo

Dado un rectángulo con base b y altura h , su área A_{rect} corresponde a:

$$A_{\text{rect}} = b \cdot h \quad (1)$$

3.2. Área de un Triángulo

1. Fórmula Estándar (Base y Altura):

$$A_{\text{tri}} = \frac{b \cdot h}{2} \quad (2)$$

2. Fórmula de Herón (Conociendo los 3 lados a, b, c):

Primero se obtiene el semiperímetro $s = \frac{a+b+c}{2}$, y luego:

$$A_{\text{Heron}} = \sqrt{s(s-a)(s-b)(s-c)} \quad (3)$$

Módulo funcional para cálculo de áreas

```

1 import math
2
3 def area_rectangulo(base: float, altura: float) -> float:
4     """Calcula el área de un rectángulo validando dimensiones
5     positivas."""
6     if base <= 0 or altura <= 0:
7         raise ValueError("Las dimensiones deben ser números
8         estrictamente positivos.")
9     return base * altura
10
11 def area_triangulo_base_altura(base: float, altura: float) -> float:
12     """Calcula el área de un triángulo con base y altura."""
13     if base <= 0 or altura <= 0:
14         raise ValueError("La base y la altura deben ser mayores que
15         cero.")
16     return (base * altura) / 2.0
17
18 def area_triangulo_heron(a: float, b: float, c: float) -> float:
19     """Calcula el área de un triángulo a partir de sus 3 lados (Fórmula
20     de Herón)."""
21     # Validación de la desigualdad triangular: la suma de 2 lados debe
22     # superar al tercero
23     if (a + b <= c) or (a + c <= b) or (b + c <= a):
24         raise ValueError("Los lados dados no forman un triángulo vá
25         lido.")
26
27     s = (a + b + c) / 2.0
28     area = math.sqrt(s * (s - a) * (s - b) * (s - c))
29     return area

```

```
25 # Pruebas con formato
26 base_r, alt_r = 12.5, 4.0
27 print(f"Rectángulo ({base_r}m x {alt_r}m) -> Área = {area_rectangulo(
    base_r, alt_r):.2f} m2")
28
29 base_t, alt_t = 10.0, 7.5
30 print(f"Triángulo (b={base_t}m, h={alt_t}m) -> Área = {
    area_triangulo_base_altura(base_t, alt_t):.2f} m2")
31
32 l1, l2, l3 = 5.0, 6.0, 7.0
33 print(f"Triángulo lados ({l1}, {l2}, {l3}) -> Área Herón = {
    area_triangulo_heron(l1, l2, l3):.4f} m2")
```

Salida en Consola

```
Rectángulo (12.5m x 4.0m) -> Área = 50.00 m2
Triángulo (b=10.0m, h=7.5m) -> Área = 37.50 m2
Triángulo lados (5.0, 6.0, 7.0) -> Área Herón = 14.6969 m2
```

4. Cálculo del Volumen de Sólidos Geométricos

El volumen cuantifica el espacio tridimensional delimitado por un cuerpo. A continuación se presentan las fórmulas analíticas de los principales sólidos geométricos:

Sólido Geométrico	Variables	Ecuación Analítica	Fórmula
Prisma Rectangular	Largo (l), Ancho (w), Alto (h)	$V = l \cdot w \cdot h$	$V = l \cdot w \cdot h$
Cilindro Recto	Radio de base (r), Altura (h)	$V = \pi r^2 h$	$V = \pi r^2 h$
Esfera	Radio (r)	$V = \frac{4}{3} \pi r^3$	$V = \frac{4}{3} \pi r^3$
Cono Circular	Radio (r), Altura (h)	$V = \frac{1}{3} \pi r^2 h$	$V = \frac{1}{3} \pi r^2 h$

Tabla 1: Fórmulas analíticas para volúmenes tridimensionales.

Implementación de volúmenes con el módulo math

```

1  import math
2
3  def volumen_prisma_rectangular(largo: float, ancho: float, alto: float
4      ) -> float:
5      """Calcula el volumen de un paralelepípedo o prisma rectangular.
6          """
7      return largo * ancho * alto
8
9  def volumen_cilindro(radio: float, altura: float) -> float:
10     """Calcula el volumen de un cilindro circular recto:  $V = \pi * r^2 * h$ . """
11     return math.pi * (radio ** 2) * altura
12
13 def volumen_esfera(radio: float) -> float:
14     """Calcula el volumen de una esfera:  $V = (4/3) * \pi * r^3$ . """
15     return (4.0 / 3.0) * math.pi * (radio ** 3)
16
17 def volumen_cono(radio: float, altura: float) -> float:
18     """Calcula el volumen de un cono circular:  $V = (1/3) * \pi * r^2 * h$ . """
19     return (1.0 / 3.0) * math.pi * (radio ** 2) * altura
20
21 # Demostración numérica
22 r_c, h_c = 3.5, 10.0
23 r_e = 4.2
24 l_p, w_p, h_p = 8.0, 5.0, 3.0
25
26 print(f"Volumen Cilindro (r={r_c}, h={h_c}): {volumen_cilindro(r_c, h_c):10.3f} u³")
27 print(f"Volumen Esfera (r={r_e}): {volumen_esfera(r_e):10.3f} u³")
28 print(f"Volumen Prisma (8x5x3): {volumen_prisma_rectangular(l_p, w_p, h_p):10.3f} u³")
29 print(f"Volumen Cono (r={r_c}, h={h_c}): {volumen_cono(r_c, h_c):10.3f} u³")

```

Salida en Consola

```
Volumen Cilindro (r=3.5, h=10.0):      384.845 u3  
Volumen Esfera (r=4.2):                310.339 u3  
Volumen Prisma (8x5x3):                120.000 u3  
Volumen Cono (r=3.5, h=10.0):         128.282 u3
```

5. Creación, Métodos y Manipulación de Listas

Las listas en Python son secuencias ordenadas, mutables y heterogéneas. Permiten almacenar colecciones dinámicas de datos.

5.1. Operaciones Fundamentales y Métodos Nativos

- `append(x)`: Añade x al final de la lista.
- `insert(i, x)`: Inserta x en la posición de índice i .
- `extend(iter)`: Concatena los elementos de otro iterable al final.
- `pop(i)`: Extrae y retorna el elemento en el índice i (último por defecto).
- `remove(x)`: Elimina la primera aparición del valor x .
- `sort()` / `reverse()`: Ordena o invierte la lista *in-place*.

Gestión completa de listas en Python

```

1 # 1. Creación e inicialización
2 frutas = ["manzana", "banana", "cereza"]
3 numeros = list(range(1, 11)) # [1, 2, 3, ..., 10]
4
5 # 2. Inserción y extensión
6 frutas.append("durazno")
7 frutas.insert(1, "arándano")
8 frutas.extend(["kiwi", "mango"])
9 print(f"Lista tras adiciones: {frutas}")
10
11 # 3. Extracción y eliminación
12 extraido = frutas.pop() # Extrae 'mango'
13 frutas.remove("banana") # Elimina 'banana'
14 print(f"Elemento extraído: {extraido} | Lista actual: {frutas}")
15
16 # 4. Slicing avanzado: lista[inicio:fin:paso]
17 sub_numeros = numeros[2:8:2] # Índices 2, 4, 6 -> [3, 5, 7]
18 invertida = numeros[::-1] # Invierte toda la lista con paso -1
19 print(f"Slicing [2:8:2]: {sub_numeros}")
20 print(f"Lista invertida: {invertida}")

```

5.2. Comprensión de Listas (*List Comprehensions*)

Permiten generar y filtrar listas de manera concisa y altamente optimizada:

[expresion for elemento in iterable if condicion]

Ejemplos prácticos de List Comprehension

```

1 # Cuadrados de números pares entre 1 y 20
2 cuadrados_pares = [x**2 for x in range(1, 21) if x % 2 == 0]
3 print(f"Cuadrados de pares: {cuadrados_pares}")
4
5 # Transformación de cadenas y filtrado por condición

```

```
6 nombres = ["ana", "pedro", "alejandro", "beatriz", "antonio", "carlos"  
7 ]  
8 nombres_con_a = [n.capitalize() for n in nombres if n.startswith("a")]  
9 print(f"Nombres con 'a' capitalizados: {nombres_con_a}")
```

Salida en Consola

```
Cuadrados de pares: [4, 16, 36, 64, 100, 144, 196, 256, 324, 400]  
Nombres con 'a' capitalizados: ['Ana', 'Alejandro', 'Antonio']
```

6. Manipulación y Procesamiento de Cadenas de Texto (*Strings*)

Las cadenas de texto en Python son secuencias **inmutables** de caracteres Unicode. Toda operación que parece modificar un string produce en realidad uno nuevo en memoria.

6.1. Métodos Principales de Procesamiento

Método	Descripción	Ejemplo
<code>.upper()</code> / <code>.lower()</code>	Convierte a mayúsculas o minúsculas	"HOLA" / "hola"
<code>.strip()</code>	Elimina espacios en blanco en extremos	"texto ".strip() → "texto"
<code>.replace(old, new)</code>	Sustituye subcadenas	".a-b".replace(" ", "/") → ".a/b"
<code>.split(sep)</code>	Divide el texto en lista según separador	"1,2,3".split(",") → ['1', '2', '3']
<code>sep.join(lista)</code>	Une elementos de lista con separador	".join(['A', 'B']) → ".A-B"
<code>.find(sub)</code>	Devuelve el índice inicial de subcadena	"hola".find("l") → 2

Tabla 2: Funciones esenciales de manipulación de cadenas en Python.

Transformación y análisis de texto

```

1  cadena_cruda = "    python para ciencia de datos y matemáticas    "
2
3  # Limpieza y formateo de títulos
4  limpia = cadena_cruda.strip()
5  titulo = limpia.title()
6  print(f"Original: '{cadena_cruda}'")
7  print(f"Limpia:    '{limpia}'")
8  print(f"Título:     '{titulo}'")
9
10 # Segmentación y construcción de slugs (Split & Join)
11 palabras = limpia.split(" ")
12 print(f"Lista de palabras: {palabras}")
13
14 slug_url = "-".join(palabras)
15 print(f"Slug URL generado: {slug_url}")
16
17 # Búsqueda, reemplazo y comprobaciones
18 texto_modificado = limpia.replace("matemáticas", "inteligencia
    artificial")
19 print(f"Reemplazo: '{texto_modificado}'")
20 print(f"¿Comienza con 'python?': {limpia.startswith('python')}")
21
22 # Inversión de un string y prueba de palíndromos
23 palindromo = "reconocer"
24 es_palindromo = palindromo == palindromo[::-1]
25 print(f"¿'{palindromo}' es palíndromo?: {es_palindromo}")

```

Salida en Consola

```

Original: '    python para ciencia de datos y matemáticas    '
Limpia:   'python para ciencia de datos y matemáticas'
Título:   'Python Para Ciencia De Datos Y Matemáticas'

```

```
Lista de palabras: ['python', 'para', 'ciencia', 'de', 'datos', 'y', 'matemáticas']
Slug URL generado: python-para-ciencia-de-datos-y-matemáticas
Reemplazo: 'python para ciencia de datos y inteligencia artificial'
¿Comienza con 'python?': True
¿'reconocer' es palíndromo?: True
```

7. Estructuras de Control Condicional (if, elif, else)

Las sentencias condicionales controlan el flujo de ejecución mediante la evaluación de predicados lógicos que devuelven True o False.

7.1. Operadores Relacionales y Lógicos

- **Relacionales:** == (igual), != (distinto), > (mayor), < (menor), >=, <=.
- **Lógicos:** and (ambas condiciones verdaderas), or (al menos una verdadera), not (inversión de verdad).
- **Pertenencia e Identidad:** in, not in, is, is not.

Evaluación condicional y clasificador académico

```
1 def clasificar_calificacion(nota: float) -> str:
2     """Clasifica el rendimiento de un estudiante según su nota."""
3     if not (0.0 <= nota <= 100.0):
4         return "Error: Calificación fuera de rango (0 - 100)."
```

5

```
6     if nota >= 90.0:
7         rendimiento = "Excelente (A)"
8     elif nota >= 80.0:
9         rendimiento = "Sobresaliente (B)"
10    elif nota >= 70.0:
11        rendimiento = "Aceptable (C)"
12    elif nota >= 60.0:
13        rendimiento = "Suficiente (D)"
14    else:
15        rendimiento = "Reprobado (F)"
16
17    return rendimiento
18
19 # Operador ternario en una línea
20 edad = 20
21 estado = "Mayor de edad" if edad >= 18 else "Menor de edad"
22 print(f"Estado: {estado}")
23
24 # Pruebas con diversas notas
25 for puntaje in [95.5, 82.0, 68.0, 54.0, 105.0]:
26     print(f"Puntaje: {puntaje:5.1f} -> {clasificar_calificacion(
27         puntaje)}")
```

Salida en Consola

```
Estado: Mayor de edad
Puntaje:  95.5 -> Excelente (A)
Puntaje:  82.0 -> Sobresaliente (B)
Puntaje:  68.0 -> Suficiente (D)
Puntaje:  54.0 -> Reprobado (F)
Puntaje: 105.0 -> Error: Calificación fuera de rango (0 - 100).
```

8. Bucles Iterativos y Generación de Patrones con for i

El bucle `for` itera secuencialmente a través de objetos iterables. La función `range(inicio, fin, paso)` genera secuencias aritméticas enteras en el rango semiabierto `[inicio, fin)`.

8.1. Generación de Patrones con Bucles Anidados

El uso de variables de iteración `for i` y bucles internos `for j` es la base para construir estructuras matriciales y patrones geométricos:

Patrones visuales con bucles for

```
1 print("--- 1. Triángulo Rectángulo de Asteriscos ---")
2 n = 5
3 for i in range(1, n + 1):
4     print("*" * i)
5
6 print("\n--- 2. Pirámide Centrada ---")
7 filas = 5
8 for i in range(1, filas + 1):
9     espacios = " " * (filas - i)
10    estrellas = "*" * (2 * i - 1)
11    print(f"{espacios}{estrellas}")
12
13 print("\n--- 3. Triángulo Numérico Creciente (Bucles Anidados) ---")
14 for i in range(1, 6):
15     for j in range(1, i + 1):
16         print(j, end=" ")
17     print()
18
19 print("\n--- 4. Rombo Simétrico ---")
20 alto = 4
21 for i in range(1, alto + 1):
22     print(" " * (alto - i) + "*" * (2 * i - 1))
23 for i in range(alto - 1, 0, -1):
24     print(" " * (alto - i) + "*" * (2 * i - 1))
```

Salida en Consola

```
--- 1. Triángulo Rectángulo de Asteriscos ---
*
**
***
****
*****

--- 2. Pirámide Centrada ---
  *
 ***
*****
*****
*****

--- 3. Triángulo Numérico Creciente (Bucles Anidados) ---
1
1 2
```

```
1 2 3
1 2 3 4
1 2 3 4 5

--- 4. Rombo Simétrico ---
  *
 ***
*****
*****
 *****
  ***
   *
```

9. Computación Científica y Matricial con NumPy

NumPy (*Numerical Python*) es la librería fundamental para la computación científica y matricial en Python. Proporciona arreglos multidimensionales homogéneos (**ndarray**) implementados en C, los cuales permiten realizar operaciones vectorizadas masivas de forma instantánea.

9.1. Creación y Estructura de Arreglos (ndarray)

Creación e inspección de arrays en NumPy

```
1 import numpy as np
2
3 # 1. Creación a partir de estructuras estándar
4 vector = np.array([10, 20, 30, 40, 50], dtype=np.float64)
5 matriz = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)
6
7 print(f"Vector: {vector} | Tipo de dato: {vector.dtype}")
8 print(f"Matriz:\n{matriz}")
9 print(f"Dimensiones: {matriz.ndim} | Forma (Shape): {matriz.shape} |
    Tamaño: {matriz.size}")
10
11 # 2. Generadores automáticos
12 ceros = np.zeros((2, 4))           # Matriz de ceros 2x4
13 unos = np.ones((3, 3))             # Matriz de unos 3x3
14 identidad = np.eye(3)              # Matriz Identidad I_3
15 rango_lineal = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
16 espaciado = np.linspace(0, 1, 5)   # 5 puntos equidistantes entre
    0 y 1
17
18 print(f"Identidad:\n{identidad}")
19 print(f"Linspace (0 a 1 en 5 pasos): {espaciado}")
```

9.2. Operaciones Vectorizadas y Álgebra Lineal

Operaciones algebraicas vectorizadas

```
1 a = np.array([1, 2, 3, 4])
2 b = np.array([10, 20, 30, 40])
3
4 # Operaciones elemento por elemento (element-wise)
5 print(f"Suma (a + b): {a + b}")
6 print(f"Multiplicación (a * b): {a * b}")
7 print(f"Potenciación (a ** 2): {a ** 2}")
8
9 # Funciones universales (ufuncs)
10 angulos = np.array([0, np.pi/6, np.pi/4, np.pi/2])
11 print(f"Seno de ángulos: {np.sin(angulos).round(4)}")
12
13 # Multiplicación matricial (Producto Punto con operador @)
14 A = np.array([[1, 2],
15               [3, 4]])
16 B = np.array([[2, 0],
17               [1, 2]])
```

```
18
19 C = A @ B # Equivalente a np.dot(A, B)
20 print(f"Producto Matricial A @ B:\n{C}")
```

Salida en Consola

```
Suma (a + b):          [11 22 33 44]
Multiplicación (a * b): [ 10 40 90 160]
Potenciación (a ** 2):  [ 1  4  9 16]
Seno de ángulos: [0.      0.5      0.7071 1.      ]
Producto Matricial A @ B:
[[ 4  4]
 [10  8]]
```

9.3. Indexación Avanzada, Slicing y Máscaras Booleanas

Filtrado booleano y sustitución

```
1 datos = np.array([15, 28, 42, 5, 89, 63, 12, 77, 34, 91])
2
3 # Máscara booleana: extraer elementos mayores a 40
4 mascara = datos > 40
5 mayores_40 = datos[mascara]
6
7 print(f"Datos originales: {datos}")
8 print(f"Máscara booleana (datos > 40): {mascara}")
9 print(f"Valores filtrados: {mayores_40}")
10
11 # Modificación condicional (sustituir valores menores a 20 por 0)
12 datos_filtrados = datos.copy()
13 datos_filtrados[datos_filtrados < 20] = 0
14 print(f"Valores con reemplazo (<20 -> 0): {datos_filtrados}")
```

9.4. Cálculos Estadísticos y de Reducción

Estadística descriptiva con NumPy

```
1 calificaciones = np.array([85.5, 90.0, 78.5, 92.0, 88.0, 65.0, 95.5,
2                             82.0])
3
4 promedio = np.mean(calificaciones)
5 desv_est = np.std(calificaciones)
6 mediana = np.median(calificaciones)
7 val_min = np.min(calificaciones)
8 val_max = np.max(calificaciones)
9 total = np.sum(calificaciones)
10
11 print(f"Total Suma: {total:.2f}")
12 print(f"Media Aritmética: {promedio:.2f}")
13 print(f"Desviación Estándar: {desv_est:.2f}")
14 print(f"Mediana: {mediana:.2f}")
15 print(f"Rango: [{val_min:.1f} - {val_max:.1f}"])
```

Salida en Consola

```
Datos originales: [15 28 42  5 89 63 12 77 34 91]
Máscara booleana (datos > 40): [False False  True False  True  True
   False  True False  True]
Valores filtrados: [42 89 63 77 91]
Valores con reemplazo (<20 -> 0): [ 0 28 42  0 89 63  0 77 34 91]
Total Suma: 676.50
Media Aritmética: 84.56
Desviación Estándar: 8.87
Mediana: 86.75
Rango: [65.0 - 95.5]
```

10. Resumen y Tabla de Referencia Rápida

Concepto / Estructura	Sintaxis Clave	Caso de Uso Principal
f-string	<code>f"{var:.2f}"</code>	Formateo legible e interpolación rápida de variables.
Funciones	<code>def func(*args, **kw):</code>	Reutilización, encapsulamiento y modularidad.
Geometría	<code>math.pi, math.sqrt</code>	Modelado analítico de áreas y volúmenes 3D.
Listas	<code>[x for x in L if cond]</code>	Almacenamiento dinámico secuencial y comprensiones.
Strings	<code>s.strip(), s.split()</code>	Limpieza, saneamiento y transformación de texto.
Condicionales	<code>if / elif / else</code>	Toma de decisiones y bifurcación lógica.
Bucles for	<code>for i in range(n):</code>	Iteración secuencial y generación de patrones gráficos.
NumPy	<code>np.array(), A @ B</code>	Computación vectorial y álgebra matricial optimizada.

Tabla 3: Resumen comparativo de los módulos de la guía.